

PROGRAMACIÓN – RETO 6

Se quiere comprobar si hay black SEO en diferentes páginas web. Esta técnica maliciosa es utilizada para manipular los resultados de los motores de búsqueda de manera no ética. Se realiza insertando contenido oculto, enlaces fraudulentos o redirigiendo a usuarios y bots de búsqueda a diferentes versiones de una página web con el fin de engañar a los motores de búsqueda

Para ello vamos a hacer un programa en Java que me compruebe varios elementos. Para ello tenemos que tener cuidado con esto:

```
<div style="display:none;">Palabras clave ocultas</div>  
<span style="font-size:0px;">Texto invisible</span>  
<a href="spamlink.com" style="visibility:hidden;">Enlace SPAM</a>
```

Explicación del funcionamiento del programa

Este programa en Java tiene como objetivo detectar posibles técnicas de Black Hat SEO dentro de un archivo HTML. Para ello, analiza el contenido del archivo línea a línea y busca ciertos estilos CSS que se utilizan habitualmente para ocultar texto o enlaces a los usuarios, pero que siguen siendo visibles para los motores de búsqueda.

Lectura del archivo HTML

El programa comienza indicando el nombre del archivo HTML que se va a analizar. Este archivo se encuentra almacenado en la carpeta del proyecto y contiene el código HTML que se quiere comprobar.

Para leer el archivo, se utiliza un `BufferedReader` junto con `FileReader`, lo que permite leer el contenido del archivo línea por línea, facilitando el análisis del texto.

Análisis del contenido

A medida que se van leyendo las líneas del archivo, el programa convierte cada línea a minúsculas para evitar problemas al comparar textos, como diferencias entre mayúsculas y minúsculas.

Después, se comprueba si cada línea contiene alguno de los siguientes patrones sospechosos: `display:none`, `font-size:0` y `visibility:hidden`. Estos estilos son típicos de técnicas de Black Hat SEO, ya que permiten introducir contenido oculto que no ve el usuario.

Detección y aviso

Cuando el programa detecta alguno de estos patrones, muestra un mensaje de advertencia por pantalla, indica el tipo de técnica encontrada y señala el número de línea del archivo donde aparece. De esta forma, el usuario puede identificar fácilmente dónde se encuentra el posible problema dentro del código HTML.

Resultado final

Si tras analizar todo el archivo no se detecta ningún patrón sospechoso, el programa muestra un mensaje indicando que no se ha encontrado Black SEO. En caso de producirse un error al leer el archivo, el programa captura la excepción y muestra un mensaje de error.

Alumna: Francisa SELMA CATALÁ

Primero DAM

RESULTADOS DE EJECUCIÓN DEL PROGRAMA

- A continuación, se ejecuta el programa utilizando el archivo index_malo.html.

En este caso, el programa detecta contenido oculto mediante estilos CSS.

Salida por consola:

ALERTA: Black SEO - texto oculto (display:none)

Linea 9

ALERTA: Black SEO - texto invisible (font-size:0)

Linea 13

ALERTA: Black SEO - enlace oculto (visibility:hidden)

Linea 17

- Posteriormente, se muestra la salida obtenida al ejecutar el programa con el archivo index_correcto.html. En este caso, no se detecta ninguna técnica de Black Hat SEO.

Salida por consola:

No se ha encontrado Black SEO.

Alumna: Francisa SELMA CATALÁ

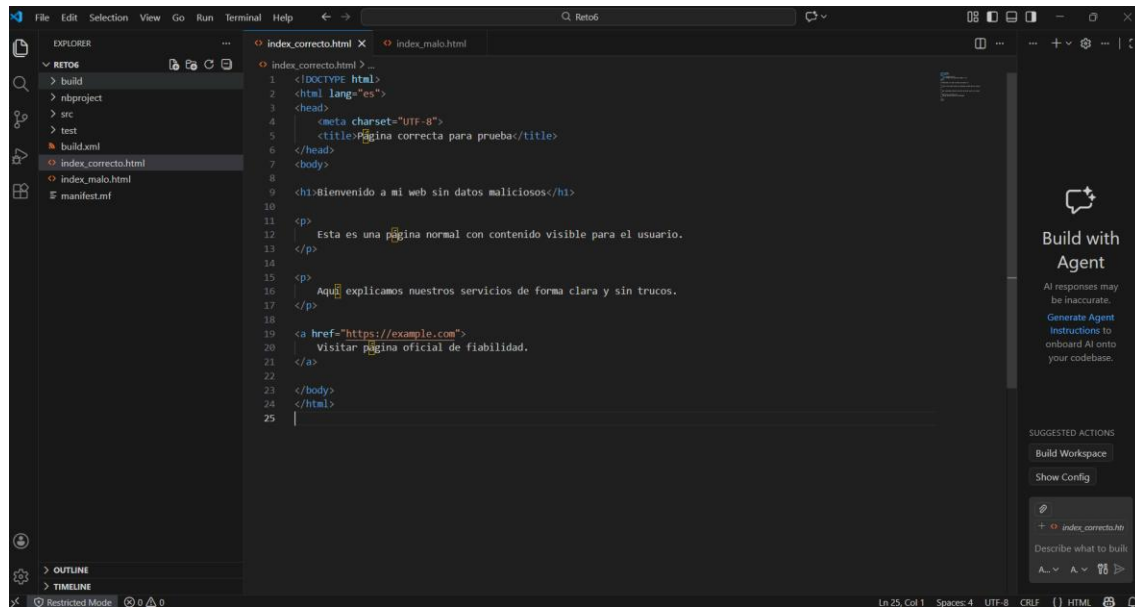
Primero DAM

Realización:

Los archivos HTML se guardan en la carpeta del proyecto de NetBeans, al mismo nivel que la carpeta src, para que puedan ser leídos correctamente por el programa Java.

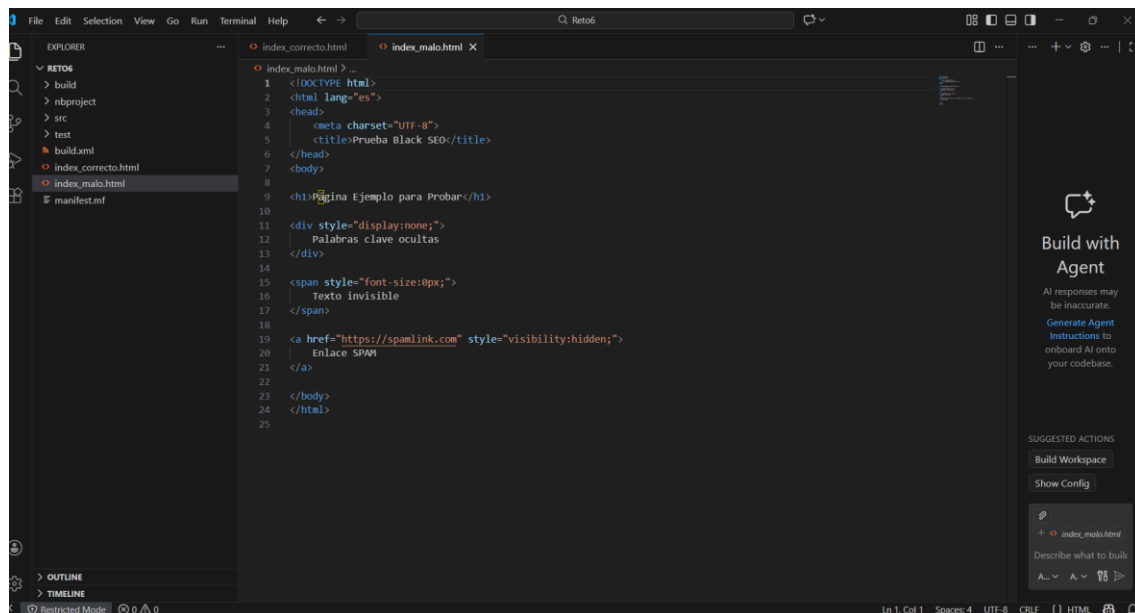
Creación de archivos html en VC.

Index_correcto.html



```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Página correcta para prueba</title>
6 </head>
7 <body>
8
9   <h1>Bienvenido a mi web sin datos maliciosos</h1>
10
11   <p>
12     Esta es una página normal con contenido visible para el usuario.
13   </p>
14
15   <p>
16     Aquí explicamos nuestros servicios de forma clara y sin trucos.
17   </p>
18
19   <a href="https://example.com">
20     Visitar página oficial de fiabilidad.
21   </a>
22
23 </body>
24 </html>
25
```

Index_malo.html



```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Prueba Black SEO</title>
6 </head>
7 <body>
8
9   <h1>Página Ejemplo para Probar</h1>
10
11   <div style="display:none;">
12     Palabras clave ocultas
13   </div>
14
15   <span style="font-size:0px;">
16     Texto invisible
17   </span>
18
19   <a href="https://spamlink.com" style="visibility:hidden;">
20     Enlace SPAM
21   </a>
22
23 </body>
24 </html>
25
```

Alumna: Francisa SELMA CATALÁ

Primero DAM

Documentos > NetBeansProjects > Reto6 >					Bus
Ordenar ~ Ver ~					
Nombre	Fecha de modificación	Tipo	Tamaño		
build	30/01/2026 22:22	Carpeta de archivos			
nbproject	30/01/2026 22:15	Carpeta de archivos			
src	30/01/2026 22:15	Carpeta de archivos			
test	30/01/2026 22:16	Carpeta de archivos			
build	30/01/2026 22:15	Microsoft Edge HT...	4 KB		
index_correcto	31/01/2026 13:14	Chrome HTML Do...	1 KB		
index_malo	30/01/2026 22:19	Chrome HTML Do...	1 KB		
manifest.mf	30/01/2026 22:15	Archivo MF	1 KB		

PROGRAMA JAVA PRINCIPAL PARA LECTURA

```
Principal.java x
Source History
1 package reto6;
2
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5 import java.io.IOException;
6
7 public class Principal {
8
9     public static void main(String[] args) {
10
11         // Archivo a analizar (cámbialo por index_correcto.html para probar el bueno)
12         String archivo = "index_malo.html";
13
14         boolean encontrado = false;
15         int numLinea = 1;
16
17         try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
18
19             String linea;
20
21             while ((linea = br.readLine()) != null) {
22
23                 String lineaMin = linea.toLowerCase();
24
25                 if (lineaMin.contains("display:none")) {
26                     System.out.println("ALERTA: Black SEO -> texto oculto (display:none)");
27                     System.out.println("Linea: " + numLinea);
28                     System.out.println("Contenido: " + linea.trim());
29                     System.out.println();
30                     encontrado = true;
31
32                 if (lineaMin.contains("font-size:0")) {
33                     System.out.println("ALERTA: Black SEO -> texto invisible (font-size:0)");
34                     System.out.println("Linea: " + numLinea);
35                     System.out.println("Contenido: " + linea.trim());
36                     System.out.println();
37                     encontrado = true;
38
39                 if (lineaMin.contains("visibility:hidden")) {
40                     System.out.println("ALERTA: Black SEO -> enlace oculto (visibility:hidden)");
41                     System.out.println("Linea: " + numLinea);
42                     System.out.println("Contenido: " + linea.trim());
43                     System.out.println();
44                     encontrado = true;
45
46                 numLinea++;
47             }
48
49             if (!encontrado) {
50                 System.out.println("No se ha encontrado Black SEO.");
51             }
52
53         } catch (IOException e) {
54             System.out.println("Error al leer el archivo: " + e.getMessage());
55         }
56     }
57 }
58
59
60
61
```

```
package reto6;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Principal {

    public static void main(String[] args) {

        // Archivo a analizar , se probará con index_malo e index_correcto.
        String archivo = "index_malo.html";

        boolean encontrado = false;
        int numLinea = 1;

        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {

            String linea;

            while ((linea = br.readLine()) != null) {

                String lineaMin = linea.toLowerCase();

                if (lineaMin.contains("display:none")) {
                    System.out.println("ALERTA: Black SEO -> texto oculto (display:none)");
                    System.out.println("Linea: " + numLinea);
                    System.out.println("Contenido: " + linea.trim());
                    System.out.println();
                    encontrado = true;
                }

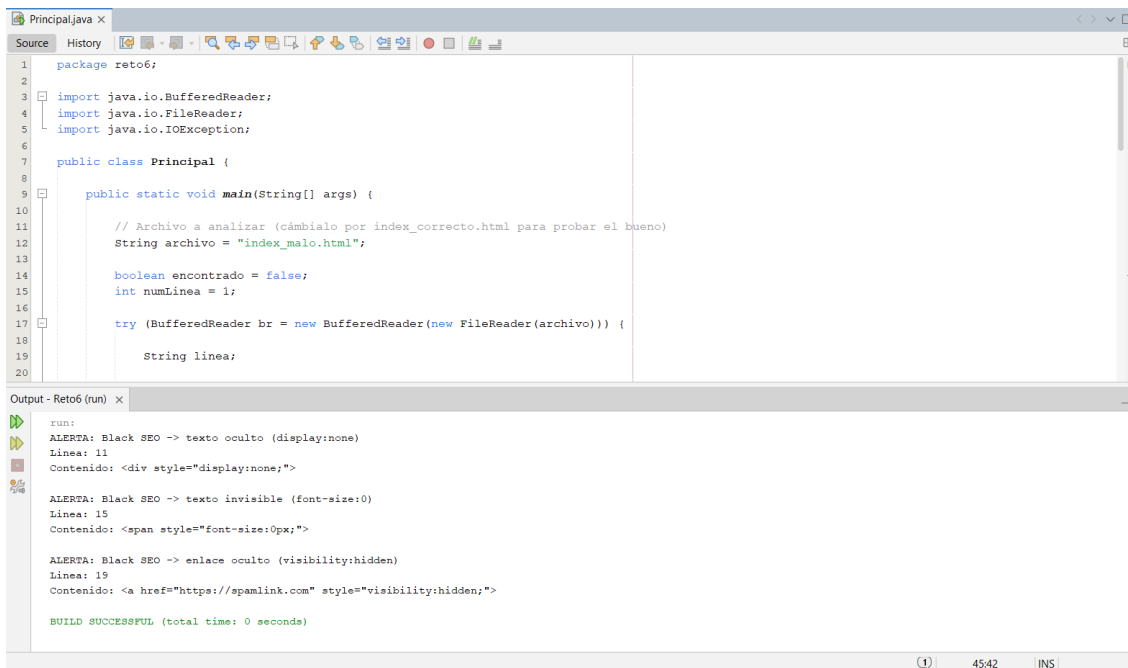
                if (lineaMin.contains("font-size:0")) {
                    System.out.println("ALERTA: Black SEO -> texto invisible (font-size:0)");
                    System.out.println("Linea: " + numLinea);
                    System.out.println("Contenido: " + linea.trim());
                    System.out.println();
                    encontrado = true;
                }

                if (lineaMin.contains("visibility:hidden")) {
                    System.out.println("ALERTA: Black SEO -> enlace oculto (visibility:hidden)");
                    System.out.println("Linea: " + numLinea);
                    System.out.println("Contenido: " + linea.trim());
                    System.out.println();
                    encontrado = true;
                }

                numLinea++;
            }
        }
    }
}
```

```
        if (!encontrado) {  
            System.out.println("No se ha encontrado Black SEO.");  
        }  
  
    } catch (IOException e) {  
        System.out.println("Error al leer el archivo: " + e.getMessage());  
    }  
}  
}
```

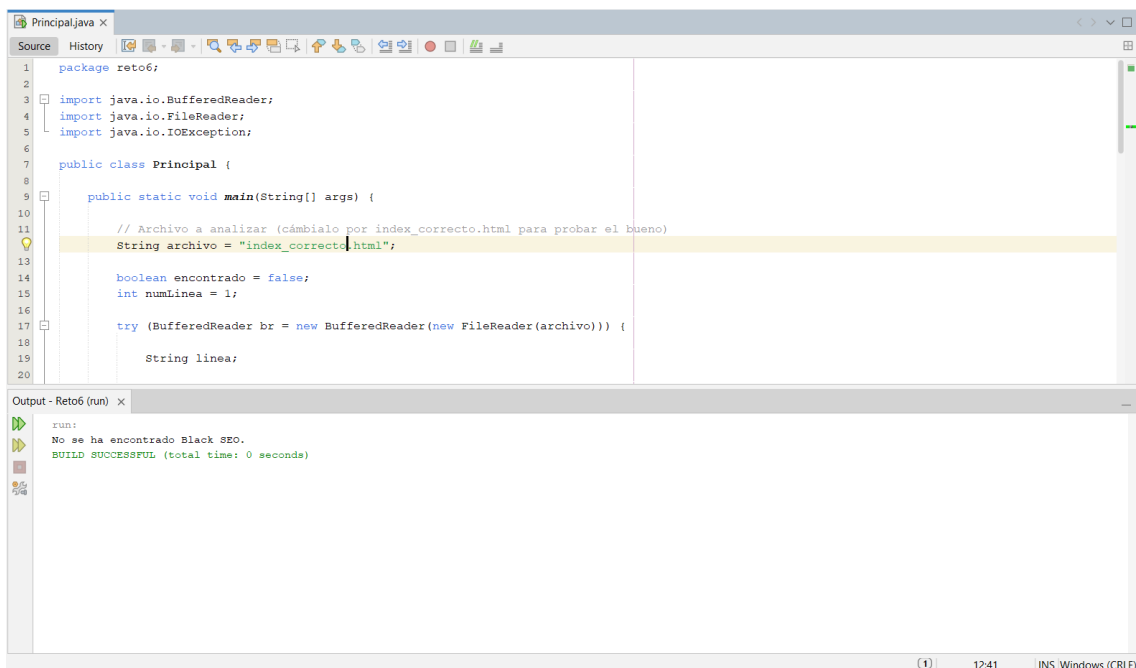
LECTURA Y COMPROBACIÓN FICHERO index_malo.html



The screenshot shows an IDE with a Java file named 'Principal.java'. The code imports `java.io.BufferedReader`, `java.io.FileReader`, and `java.io.IOException`. It defines a `Principal` class with a `main` method. Inside `main`, it sets `archivo = "index_malo.html"`, initializes `boolean encontrado = false` and `int numLinea = 1`. It then enters a `try` block to read the file line by line. The output window shows three alerts: 'ALERTA: Black SEO -> texto oculto (display:none)' at line 11, 'ALERTA: Black SEO -> texto invisible (font-size:0)' at line 15, and 'ALERTA: Black SEO -> enlace oculto (visibility:hidden)' at line 19. The build is successful.

```
package reto6;  
  
import java.io.BufferedReader;  
import java.io.FileReader;  
import java.io.IOException;  
  
public class Principal {  
  
    public static void main(String[] args) {  
  
        // Archivo a analizar (cámbialo por index_correcto.html para probar el bueno)  
        String archivo = "index_malo.html";  
  
        boolean encontrado = false;  
        int numLinea = 1;  
  
        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {  
  
            String linea;  
  
            run:  
            ALERTA: Black SEO -> texto oculto (display:none)  
            Línea: 11  
            Contenido: <div style="display:none;">  
  
            ALERTA: Black SEO -> texto invisible (font-size:0)  
            Línea: 15  
            Contenido: <span style="font-size:0px;">  
  
            ALERTA: Black SEO -> enlace oculto (visibility:hidden)  
            Línea: 19  
            Contenido: <a href="https://spamlink.com" style="visibility:hidden;">  
  
            BUILD SUCCESSFUL (total time: 0 seconds)
```

LECTURA Y COMPROBACIÓN FICHERO index_correcto.html



The screenshot shows the same IDE with the same Java code, but the `archivo` variable is now set to `"index_correcto.html"`. The output window shows a single message: 'No se ha encontrado Black SEO.' The build is successful.

```
package reto6;  
  
import java.io.BufferedReader;  
import java.io.FileReader;  
import java.io.IOException;  
  
public class Principal {  
  
    public static void main(String[] args) {  
  
        // Archivo a analizar (cámbialo por index_correcto.html para probar el bueno)  
        String archivo = "index_correcto.html";  
  
        boolean encontrado = false;  
        int numLinea = 1;  
  
        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {  
  
            String linea;  
  
            run:  
            No se ha encontrado Black SEO.  
  
            BUILD SUCCESSFUL (total time: 0 seconds)
```